EXPERIMENT-01

Question:

A class consists of 40 students who have secured different grades in an examination. The grades are classified as F, E, D, C, B, A, and S, where F represents the lowest grade and S represents the highest grade. The number of students who secured each grade is given below.

Grade	Number of Students
F	3
E	4
D	5
C	8
B	9
A	7
S	4
Total	40

Develop a Python program to store the grades and corresponding student counts, draw a pie chart showing percentage distribution with grade labels and percentages, draw a bar chart comparing student counts with axis labels and exact values above each bar, add suitable titles and legends, identify the maximum and minimum grades, and calculate the combined percentage of students who secured grades A and S.

Aim:

To write a Python program using Matplotlib to visualize the grade distribution of 40 students using a pie chart and a bar chart, identify the grades with the maximum and minimum student counts, and calculate the combined percentage of students who secured A and S grades.

Algorithm:

1. Start the program.
2. Import the Matplotlib library.
3. Store the grades and their corresponding number of students in two lists.

4. Calculate the total number of students.
5. Draw a pie chart using the student counts, grade labels, and percentage values.
6. Add a suitable title and legend to the pie chart, and display it.
7. Draw a bar chart using the grades and student counts.
8. Label the X-axis as Grades and the Y-axis as Number of Students.
9. Display the exact student count above each bar and add a title and legend.
10. Find the maximum and minimum student counts and identify their corresponding grades.
11. Add the numbers of students who secured A and S grades and calculate their combined percentage.
12. Display the maximum grade, minimum grade, and combined percentage.
13. Stop the program.

Program:

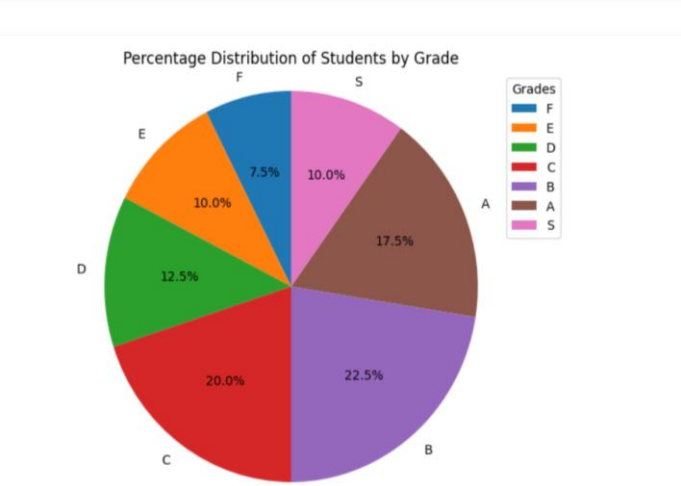
```
import matplotlib.pyplot as plt
# Store grades and number of students
grades = ['F', 'E', 'D', 'C', 'B', 'A', 'S']
students = [3, 4, 5, 8, 9, 7, 4]
# Calculate total number of students
total = sum(students)
# Pie chart
plt.figure(
figsize=(8, 6))
plt.pie(students,
    labels=grades,
    autopct='%1.1f%%',
    startangle=90
)
plt.title("Percentage Distribution of Students by Grade")
plt.legend(grades, title="Grades")
plt.axis('equal')
plt.show()
# Bar chart
```

```
plt.figure(figsize=(8, 6))
bars = plt.bar(
grades,
    students,
    color='skyblue',
    edgecolor='black',
    label='Number of Students'
)
plt.xlabel("Grades")
plt.ylabel("Number of Students")
plt.title("Number of Students in Each Grade")
plt.legend()
# Display exact number above each bar
for bar in bars:
    height = bar.get_height()
    plt.text(
        bar.get_x() + bar.get_width() / 2,
        height,
        str(int(height)),
        ha='center',
        va='bottom'
    )
plt.show()
# Identify maximum and minimum grades
max_count = max(students)
min_count = min(students)
max_grade = grades[students.index(max_count)]
min_grade = grades[students.index(min_count)]
print("Maximum number of students:")
print("Grade:", max_grade)
print("Number of students:", max_count)
print("\nMinimum number of students:")
print("Grade:", min_grade)
```

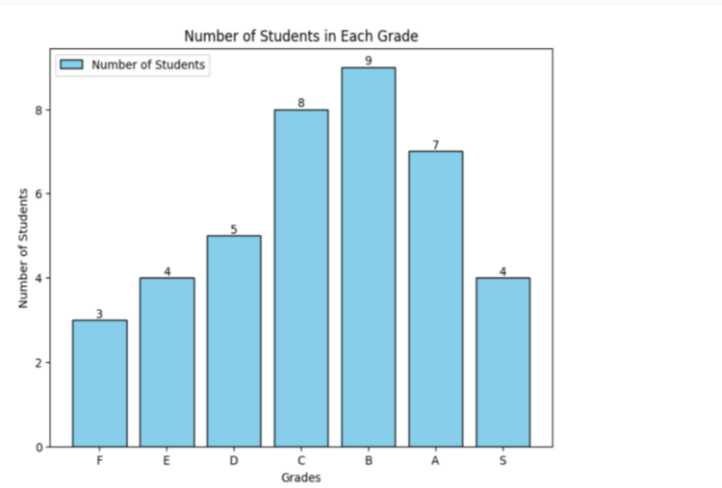
```
print("Number of students:", min_count)
# Calculate combined percentage of A and S
a_students = students[grades.index('A')]
s_students = students[grades.index('S')]
combined_students = a_students + s_students
combined_percentage = (combined_students / total) * 100
print("\nStudents who secured A grade:", a_students)
print("Students who secured S grade:", s_students)
print("Combined number of students:", combined_students)
print("Combined percentage of A and S grades:",
    round(combined_percentage, 2), "%")
```

Output:

1. Pie chart showing the percentage distribution of students across grades F, E, D, C, B, A, and S.



2. Bar chart comparing the number of students in each grade, with the exact count shown above each bar.



Console output:

Maximum number of students:

Grade: B

Number of students: 9

Minimum number of students:

Grade: F

Number of students: 3

Students who secured A grade: 7

Students who secured S grade: 4

Combined number of students: 11

Combined percentage of A and S grades: 27.5 %

Result:

Thus, the Python program was developed and executed successfully to visualize the grade distribution using pie and bar charts. Grade B has the maximum number of students (9), Grade F has the minimum number of students (3), and 27.5% of the students secured A and S grades combined.

EXPERIMENT-02

Question:

Semantic Network Using NetworkX

Aim: To represent concepts and labeled relationships as a directed semantic network.

Software Required: Python 3.x and NetworkX (install with: pip install networkx)

Procedure / Algorithm

1. Create a directed graph.

2. Add concepts as nodes implicitly through edges.

3. Add labeled relationships.

4. Display each relationship.

Program

import networkx as nx

G = nx.DiGraph()

G.add_edge("Dog", "Mammal", relation="is-a")

G.add_edge("Mammal", "Animal", relation="is-a")

G.add_edge("Dog", "Tommy", relation="has-instance")

print("Semantic Network Relationships:")

for source, target, data in G.edges(data=True):

print(source, "--", data["relation"], "-->", target)

Sample Input

Concepts and relations are predefined in the program.

Expected Output

Semantic Network Relationships:

Dog -- is-a --> Mammal

Dog -- has-instance --> Tommy

Mammal -- is-a --> Animal

Result: The semantic network represented concepts and their relationships.

Aim:

To represent concepts and their labeled relationships as a directed semantic network using Python and the NetworkX library.

Algorithm:

1. Start the program.
2. Import the NetworkX library.
3. Create a directed graph using nx.DiGraph().
4. Add an edge from Dog to Mammal with the relationship is-a.
5. Add an edge from Mammal to Animal with the relationship is-a.
6. Add an edge from Dog to Tommy with the relationship has-instance.
7. Display the heading Semantic Network Relationships.
8. Traverse all edges and display the source, relationship label, and target.
9. Stop the program.

Program:

```
import networkx as nx
# Create a directed graph
G = nx.DiGraph()
# Add concepts and labeled relationships
G.add_edge("Dog", "Mammal", relation="is-a")
G.add_edge("Mammal", "Animal", relation="is-a")
G.add_edge("Dog", "Tommy", relation="has-instance")
# Display relationships
print("Semantic Network Relationships:")
for source, target, data in G.edges(data=True):
print(source, "--", data["relation"], "-->", target)
```

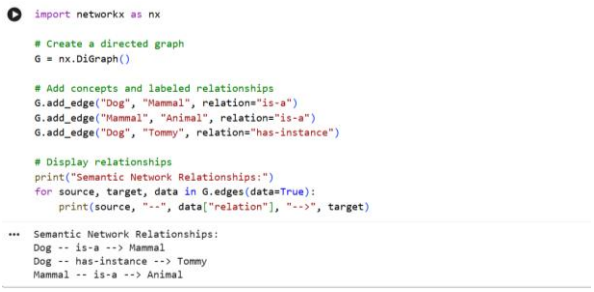
Output:

Semantic Network Relationships:

Dog -- is-a --> Mammal

Dog -- has-instance --> Tommy

Mammal -- is-a --> Animal



```
import networkx as nx

# Create a directed graph
G = nx.DiGraph()

# Add concepts and labeled relationships
G.add_edge("Dog", "Mammal", relation="is-a")
G.add_edge("Mammal", "Animal", relation="is-a")
G.add_edge("Dog", "Tommy", relation="has-instance")

# Display relationships
print("Semantic Network Relationships:")
for source, target, data in G.edges(data=True):
    print(source, "--", data["relation"], "-->", target)
```

```
Semantic Network Relationships:
Dog -- is-a --> Mammal
Dog -- has-instance --> Tommy
Mammal -- is-a --> Animal
```

Result:

Thus, the semantic network was represented successfully using a directed graph in NetworkX, and all concepts and their labeled relationships were displayed.

EXPERIMENT-02

Question:

Frame-Based Knowledge Representation

Aim: To represent an object as a frame containing slots and values.

Software Required: Python 3.x

Procedure / Algorithm

1. Create a dictionary to represent a frame.

2. Use keys as slots and values as slot values.

3. Display the frame.

Program

vehicle = {

"class": "Vehicle",

"brand": "Toyota",

"fuel_type": "Petrol",

"wheels": 4,

"color": "White"}

print("Vehicle Frame")

for slot, value in vehicle.items():

print(slot, ":", value)

Sample Input

Frame values are predefined in the program.

Expected Output

Vehicle Frame

class : Vehicle

brand : Toyota

fuel_type : Petrol

wheels : 4

color : White

Result: The vehicle was represented using slots and their corresponding values.

Aim:

To represent a vehicle as a frame using a Python dictionary containing slots and their corresponding values.

Algorithm:

1. Start the program.
2. Create a dictionary named vehicle to represent the frame.
3. Use the keys class, brand, fuel_type, wheels, and color as slots.
4. Assign the corresponding values to each slot.
5. Display the heading Vehicle Frame.
6. Traverse the dictionary using a for loop.
7. Print each slot and its value.
8. Stop the program.

Program

```
# Create a frame using a dictionary
vehicle = {
"class": "Vehicle",
"brand": "Toyota",
"fuel_type": "Petrol",
"wheels": 4,
"color": "White"
}
# Display the frame
print("Vehicle Frame")
for slot, value in vehicle.items():
print(slot, ":", value)
```

Output

Vehicle Frame

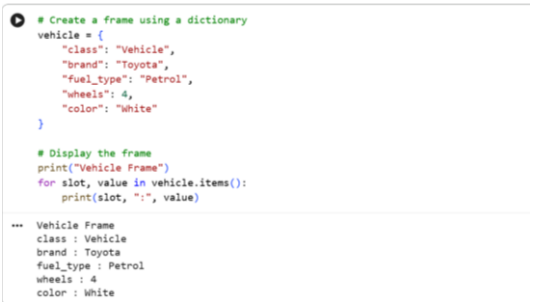
class : Vehicle

brand : Toyota

fuel_type : Petrol

wheels : 4

color : White



```
# Create a frame using a dictionary
vehicle = {
    "class": "Vehicle",
    "brand": "Toyota",
    "fuel_type": "Petrol",
    "wheels": 4,
    "color": "White"
}

# Display the frame
print("Vehicle Frame")
for slot, value in vehicle.items():
    print(slot, ":", value)
```

```
Vehicle Frame
class : Vehicle
brand : Toyota
fuel_type : Petrol
wheels : 4
color : White
```

Result:

Thus, the vehicle was successfully represented using a frame-based knowledge representation, with each slot storing its corresponding value.